The reason for this difference is historical. Internet Explorer 8 was released in 2009, when the CORS spec was still young. The Internet Explorer developers wanted to ensure that cross-origin requests were done in a safe manner, so they limited how cross-origin requests could be made. As the CORS spec has evolved, the kinks have been worked out to the point where Internet Explorer 10 and above have support for all CORS functionality. But Internet Explorer 8 and Internet Explorer 9 represent a large proportion of the browsers still in use, so they can't be ignored. Here is a look at how the `XDomainRequest` object is different from `XMLHttpRequest`.

GET AND POST ONLY

The `XDomainRequest` object can only make HTTP requests using `GET` or `POST`. Other HTTP methods such as `HEAD`, `PUT`, or `DELETE` aren't allowed.

LIMITED SCHEME SUPPORT

`XDomainRequest` only supports CORS from the http:// or https:// schemes. This means that opening a file on your computer won't work, because this uses the file:// scheme. Furthermore, HTTP pages can only make CORS requests to other HTTP pages, and HTTPS pages can only make CORS requests to other HTTPS pages.

CONTENT-TYPE TEXT/PLAIN ONLY FOR REQUESTS

The `XDomainRequest` object can only make requests with the `text/plain` Content-Type. Content-Type identifies the data type of the HTTP body. It's an HTTP header that can be present on both HTTP requests and HTTP responses. When visiting a web page, for example, the HTML file has a Content-Type of `text/html`, and a JPEG image has a Content-Type of `image/jpeg`. The Content-Type serves as a hint to browsers and servers about what data type to expect. If your server expects a different content type on requests, it will have to be modified to also allow the `text/plain` `Content-Type`.

ASYNCHRONOUS REQUESTS ONLY

The `XDomainRequest` object supports only asynchronous requests; there is no way to use it to make synchronous requests. But this is a feature that won't be missed. The blocking nature of synchronous requests can lead to a frustrating user experience, because the user can't interact with the page until the request completes.

NO CUSTOM REQUEST HEADERS

Earlier I showed how the `XMLHttpRequest` object uses the `setRequestHeader` method to include custom request headers on the request. The `XDomainRequest` object doesn't have a `setRequestHeader` function, and doesn't allow custom request headers on requests.

NO COOKIES OR USER CREDENTIALS

The `XDomainRequest` object never includes cookies or other user credentials in requests. Furthermore, there is no `withCredentials` property to override behavior. Cookies are the main mechanism for identifying visitors to a website, which means the `XDomainRequest` object is best suited for making requests to public data that doesn't serve any user-specific information.